Task 3: Time Series Forecasting

Name: SUPRAJEET VIJAY PATIL

Matriculation Number: 100004941

Date of Submission: 27.01.2026

INTRODUCTION

Time series forecasting is a critical analytical task in data science and artificial intelligence, focusing on predicting future values based on previously observed data points ordered in time. This task is particularly important in economic and business domains, where accurate forecasting of indicators such as sales, demand, and revenue directly influences strategic planning, policy formulation, and resource allocation. In this context, Task 3: Time Series Forecasting aims to design, implement, and evaluate advanced forecasting models capable of learning complex temporal patterns from historical sales data.

In this study, we analyze and forecast U.S. sales data using three advanced RNN architectures: Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Bidirectional LSTM (Bi-LSTM). These models are designed to address the limitations of standard RNNs by effectively learning long-term dependencies and reducing issues such as vanishing gradients.

The objectives of this work are:

- To compare the performance of LSTM, GRU, and Bi-LSTM models on historical sales data.
- To generate real future forecasts for the next six months.
- To analyze model reliability using residual (error) distribution analysis.

The forecasting framework employs a sliding window approach, where past observations are used as input to predict future values. Models are trained on normalized data and evaluated through visual comparison with actual sales values, multi-step future forecasting, and error distribution analysis. This comprehensive evaluation ensures not only predictive accuracy but also model robustness and stability. By implementing the PGD attack directly on top of a CNN classifier, this work bridges theoretical concepts presented in the lecture notes with practical experimentation. The results provide insight into the robustness properties of neural networks and demonstrate why adversarial robustness must be considered an integral component of modern machine learning system design rather than an optional enhancement.

By leveraging deep learning–based sequence models, this research demonstrates a robust and scalable approach to sales forecasting that overcomes the limitations of traditional methods and provides actionable insights for economic analysis and decision-making.

Overall, **Task 3: Time Series Forecasting** demonstrates the effectiveness of deep recurrent neural networks in handling complex, real-world time series data. The findings of this task highlight the advantages of bidirectional and gated recurrent architectures over traditional methods and provide valuable insights into the application of deep learning techniques for economic forecasting.

METHODOLOGY

This task follows a structured methodology to design, train, and evaluate deep learning models for time series forecasting. The overall process consists of data preparation, model construction, training, future forecasting, and performance evaluation. Each step was carefully implemented to ensure fair comparison between models and reliable forecasting results.

Step 1: Data Collection and Understanding:

The dataset used in this task contains monthly U.S. sales values recorded over multiple years. Each record consists of an observation date and a corresponding total sales value (TOTALSA). Since the data is indexed by time, it naturally forms a univariate time series.

Before modeling, the dataset was visually inspected to understand its behavior. The sales values show clear long-term trends and seasonal variations, indicating that simple linear models would be insufficient. These characteristics motivated the use of recurrent neural network–based models, which are well suited for capturing temporal dependencies in sequential data.

Data Preprocessing:

To prepare the data for neural network training, several preprocessing steps were applied:

1. **Type Conversion:**
   The sales values were converted to floating-point format to ensure numerical consistency during training.
2. **Normalization:**
   Min–Max normalization was applied to scale the sales values into the range [0, 1]. This step is essential for neural networks, as it prevents large numerical values from dominating the learning process and improves convergence speed.
3. **Train–Test Split:**
   The dataset was split chronologically, with 80% of the data used for training and the remaining 20% reserved for testing. This approach preserves the temporal order of the data and avoids information leakage from future observations into the training set.

Sliding Window Construction:

Since recurrent neural networks require fixed-length input sequences, a sliding window approach was adopted. A look-back period of 12 months was selected so that each input sequence represents one full year of historical data. For each time step:

- The previous 12 months of sales values are used as input.
- The sales value of the following month is used as the target output.

This design allows the models to learn yearly seasonal patterns while still capturing longer-term trends. The generated input data was then reshaped into the format required by recurrent neural networks.

Model Architecture Design:

Three different recurrent neural network architectures were implemented and compared:

Long Short-Term Memory (LSTM):
The LSTM model uses memory cells and gating mechanisms to control information flow across time steps. This structure allows the model to retain relevant historical information and discard irrelevant data, making it suitable for time series with long-term dependencies.

Gated Recurrent Unit (GRU):
The GRU model is a simplified version of LSTM that combines the forget and input gates into a single update gate. This results in fewer parameters and faster training while maintaining strong predictive performance.

Bidirectional LSTM (Bi-LSTM):
The Bi-LSTM model processes the input sequence in both forward and backward directions. This enables the model to learn richer temporal representations by considering patterns that may not be captured when processing data in a single direction.

Each model consists of:

- A single recurrent layer with 50 hidden units
- A fully connected output layer with one neuron
- Mean Squared Error as the loss function
- Adam optimizer for weight updates

The same architecture depth and training configuration were used for all models to ensure a fair comparison.

Model Training:
All models were trained using the training dataset for 20 epochs with a batch size of 16. The training process was kept identical across models to avoid bias in performance comparison. During training, the models learned to minimize the difference between predicted and actual sales values.After training, predictions were generated for both the training and test datasets. The predicted values were then transformed back to the original scale using the inverse of the normalization process.

Future Forecasting Strategy:
To evaluate the practical usefulness of the models, real future forecasting was performed. The last 12 months of available data were used as the initial input window. Each model predicted the next month's sales value, which was then appended to the input window to predict subsequent months.

This iterative process was repeated to generate forecasts for the next six months. This approach closely reflects real-world forecasting scenarios, where future values are unknown and must be predicted sequentially based on previous predictions.

PYTHON IMPLEMENTATION

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, GRU, Bidirectional

# ---------------------------------------------------------
# 1. SETUP DATA
# ---------------------------------------------------------
filename = '/content/TOTALSA.csv'
print(f"Loading dataset: {filename}...")
df = pd.read_csv(filename, parse_dates=['observation_date'])

# Use the 'TOTALSA' column
data_values = df['TOTALSA'].values.astype('float32').reshape(-1, 1)

# Normalize
scaler = MinMaxScaler(feature_range=(0, 1))
dataset = scaler.fit_transform(data_values)

# Split 80% Train, 20% Test
train_size = int(len(dataset) * 0.80)
train, test = dataset[0:train_size, :], dataset[train_size:len(dataset), :]

# Helper to create windows
def create_dataset(dataset, look_back=12):
    dataX, dataY = [], []
    for i in range(len(dataset) - look_back - 1):
        dataX.append(dataset[i:(i + look_back), 0])
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)

look_back = 12
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)

# Reshape for RNN
trainX = np.reshape(trainX, (trainX.shape[0], 1, trainX.shape[1]))
testX = np.reshape(testX, (testX.shape[0], 1, testX.shape[1]))
```

```python
# ---------------------------------------------------------
# 2. TRAIN ALL 3 MODELS
# ---------------------------------------------------------
models_dict = {}
predictions_dict = {}
model_names = ['LSTM', 'GRU', 'Bi-LSTM']

print("Starting training for all models...")

for name in model_names:
    print(f"Training {name}...")
    model = Sequential()

    if name == 'LSTM':
        model.add(LSTM(50, input_shape=(1, look_back)))
    elif name == 'GRU':
        model.add(GRU(50, input_shape=(1, look_back)))
    elif name == 'Bi-LSTM':
        model.add(Bidirectional(LSTM(50), input_shape=(1, look_back)))

    model.add(Dense(1))
    model.compile(loss='mean_squared_error', optimizer='adam')

    # Train
    model.fit(trainX, trainY, epochs=20, batch_size=16, verbose=0)

    # Store the trained model in the dictionary
    models_dict[name] = model

    # Predict (Train and Test)
    trainP = model.predict(trainX)
    testP = model.predict(testX)

    # Invert and Store
    predictions_dict[name] = {
        'train': scaler.inverse_transform(trainP),
        'test': scaler.inverse_transform(testP)
    }

# ---------------------------------------------------------
# 3. PREPARE PLOTTING DATA
# ---------------------------------------------------------
# Inverse transform actual data for plotting
actual_data_inv = scaler.inverse_transform(dataset)

# create empty arrays to align plots with the timeline
def align_predictions(original_data, train_preds, test_preds, look_back):
    # Placeholders filled with NaN
    train_plot = np.empty_like(original_data)
    train_plot[:, :] = np.nan
    test_plot = np.empty_like(original_data)
    test_plot[:, :] = np.nan

    # Fill Train
    train_plot[look_back:len(train_preds)+look_back, :] = train_preds

    # Fill Test
    # Calculate start index based on training length
    start_test = len(train_preds) + (look_back * 2) + 1
    # Handle array size mismatch if any (slice to fit)
    limit = len(original_data) - start_test
    if len(test_preds) > limit:
        test_preds = test_preds[:limit]
```

```python
    test_plot[start_test:start_test+len(test_preds), :] = test_preds

    return train_plot, test_plot

# ---------------------------------------------------------
# 4. VISUALIZATION (BARS + 3 MODEL LINES)
# ---------------------------------------------------------
plt.figure(figsize=(16, 9))

# 1. SETUP ZOOM (Last 5 Years = 60 Months)
# This fits best on a screen. If you want 10 years, change this to 10.
zoom_years = 5
zoom_idx = zoom_years * 12

# Get the dates and actual values for this period
dates_zoom = df['observation_date'][-zoom_idx:]
actual_zoom = actual_data_inv[-zoom_idx:].flatten() # Flatten for bar chart

# 2. PLOT ACTUAL DATA AS BARS
plt.bar(dates_zoom, actual_zoom, label='Actual Sales (Bars)', color='skyblue', alpha=0.5, width=20)

# 3. PLOT ALL 3 MODELS AS LINES
# We define specific colors and marker shapes to tell them apart easily
styles = {
    'LSTM':    {'color': 'blue',  'marker': 'o', 'linestyle': '--'},
    'GRU':     {'color': 'green', 'marker': 's', 'linestyle': '-.'},
    'Bi-LSTM': {'color': 'red',   'marker': '^', 'linestyle': '-'}
}

for name in model_names:
    # Get the test predictions for this model
    test_pred_full = predictions_dict[name]['test']

    # Extract only the last 60 months to match the zoom
    # We use [-zoom_idx:] to get the end of the array
    pred_zoom = test_pred_full[-zoom_idx:].flatten()

    # Plot the line
    plt.plot(dates_zoom, pred_zoom, label=f'{name} Prediction',
             color=styles[name]['color'],
             marker=styles[name]['marker'],
             linestyle=styles[name]['linestyle'],
             linewidth=2, markersize=5)

# 4. FORMATTING
plt.title(f'Comparison: Actual vs 3 Models (Last {zoom_years} Years)', fontsize=16)
plt.xlabel('Date', fontsize=12)
plt.ylabel('Total Sales', fontsize=12)
plt.legend(loc='upper left', frameon=True, shadow=True)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()

plt.show()
```

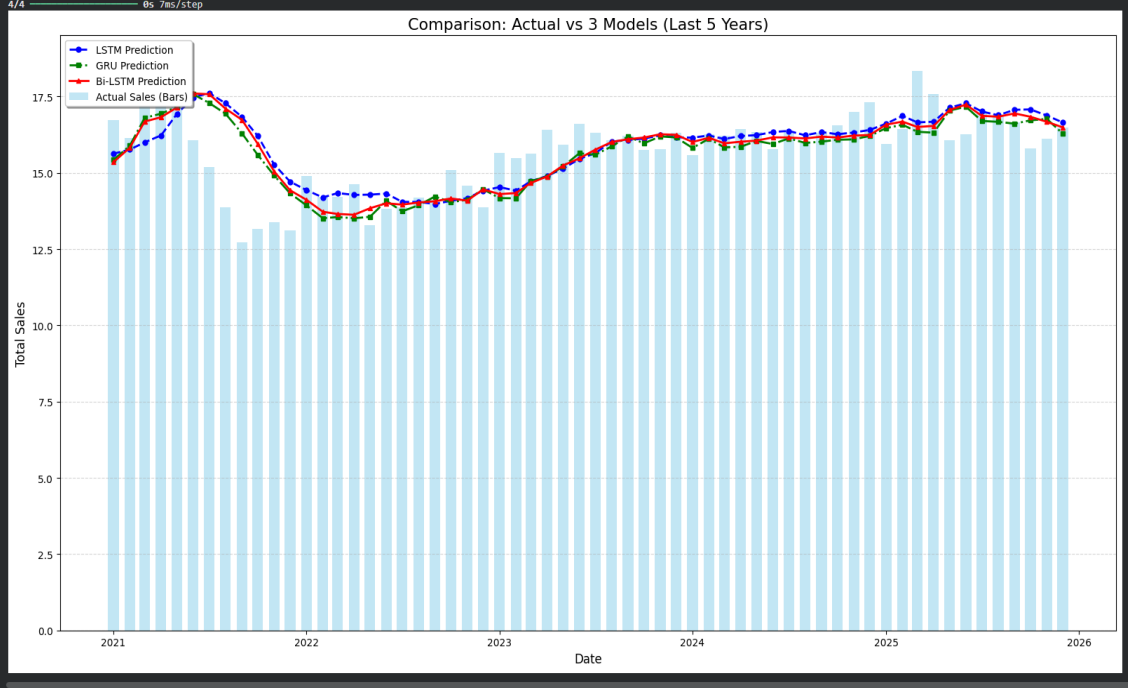
```python
# -------------------------------------------------------
#  COMPARING FUTURE PREDICTIONS OF ALL 3 MODELS
# -------------------------------------------------------

future_predictions_all_models = {}

print("\nGenerating future predictions for all models...")

for model_name in model_names:
    model = models_dict[model_name]

    # Start with the last known data sequence
    last_sequence = dataset[-look_back:]
    curr_seq = last_sequence.reshape((1, 1, look_back))

    current_model_future_preds = []
    for i in range(6): # Predict next 6 months
        # Predict one step
        pred = model.predict(curr_seq, verbose=0)
        current_model_future_preds.append(pred[0,0])

        # Update sequence: remove first val, add new prediction at end
        new_seq_val = pred.reshape(1, 1, 1)
        curr_seq = np.append(curr_seq[:, :, 1:], new_seq_val, axis=2)

    # Invert scaling and store
    future_predictions_all_models[model_name] = scaler.inverse_transform(np.array(current_model_future_preds).reshape(-1, 1))

# Generate future dates for plotting
last_date = df['observation_date'].iloc[-1]
future_dates = pd.date_range(start=last_date + pd.DateOffset(months=1), periods=6, freq='MS')

# Prepare data for grouped bar chart
predictions_df = pd.DataFrame({
    'Date': future_dates,
    'LSTM': future_predictions_all_models['LSTM'].flatten(),
    'GRU': future_predictions_all_models['GRU'].flatten(),
    'Bi-LSTM': future_predictions_all_models['Bi-LSTM'].flatten()
})

# Plotting
plt.figure(figsize=(14, 8))

bar_width = 0.2
index = np.arange(len(future_dates))

plt.bar(index, predictions_df['LSTM'], bar_width, label='LSTM Prediction', color=styles['LSTM']['color'], alpha=0.8)
plt.bar(index + bar_width, predictions_df['GRU'], bar_width, label='GRU Prediction', color=styles['GRU']['color'], alpha=0.8)
plt.bar(index + 2 * bar_width, predictions_df['Bi-LSTM'], bar_width, label='Bi-LSTM Prediction', color=styles['Bi-LSTM']['color'], alpha=0.8)

plt.xlabel('Date', fontsize=12)
plt.ylabel('Total Sales (Million Units)', fontsize=12)
plt.title('Future Sales Forecast Comparison Across Models (Next 6 Months)', fontsize=16)
plt.xticks(index + bar_width, [date.strftime('%Y-%m') for date in future_dates], rotation=45, ha='right')
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```python
# ---------------------------------------------------------
#  VISUALIZATION 2: ERROR DISTRIBUTION (ALL 3 MODELS)
# ---------------------------------------------------------
plt.figure(figsize=(12, 7))

residuals_list = []
labels_list = []
# Colors matching the line chart above
colors_hist = ['blue', 'green', 'red']

# The actual values corresponding to the test predictions are in testY
actual_test_values_inv = scaler.inverse_transform(testY.reshape(-1, 1))

# Calculate the dates corresponding to the testY values for year information
# testY starts at index 'look_back' of the 'test' array, which corresponds to
# original_df_index = train_size + look_back.
# The length of testY is len(test) - look_back - 1.
test_y_start_idx_df = train_size + look_back
test_y_end_idx_df = train_size + len(test) - 1 # Exclusive for iloc slicing
test_dates_for_errors = df['observation_date'].iloc[test_y_start_idx_df : test_y_end_idx_df]

min_year = test_dates_for_errors.dt.year.min()
max_year = test_dates_for_errors.dt.year.max()
year_range_str = f"({min_year}-{max_year})"

for name in model_names:
    # Get the test predictions for the current model
    model_test_predictions = predictions_dict[name]['test']

    # Error = Actual - Predicted
    error = actual_test_values_inv - model_test_predictions
    residuals_list.append(error.flatten())
    labels_list.append(name)

# Plot grouped histogram
plt.hist(residuals_list, bins=15, color=colors_hist, label=labels_list,
         edgecolor='black', alpha=0.7)

plt.axvline(0, color='black', linestyle='--', linewidth=2, label='Perfect Prediction')
plt.title(f'Error Distribution Comparison (Zero is best) {year_range_str}', fontsize=16)
plt.xlabel('Error (Actual Sales - Predicted Sales)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

OUTPUT IMAGES

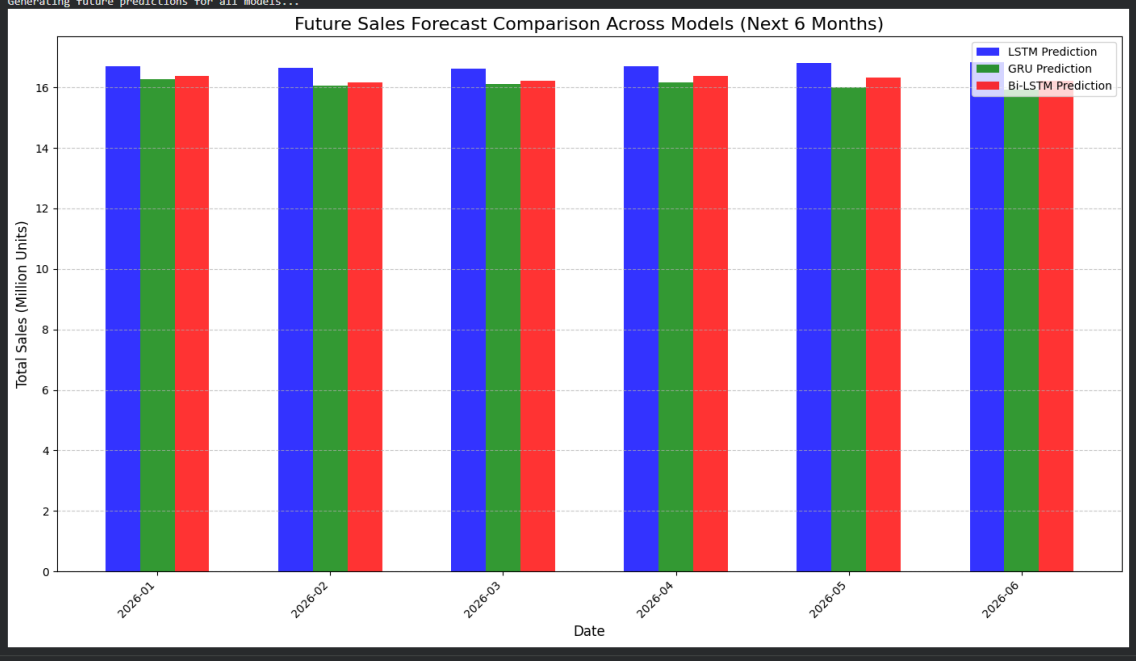
```
plt.tight_layout()

plt.show()
```

```
Training Bi-LSTM...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/bidirectional.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as
  super().__init__(**kwargs)
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
```

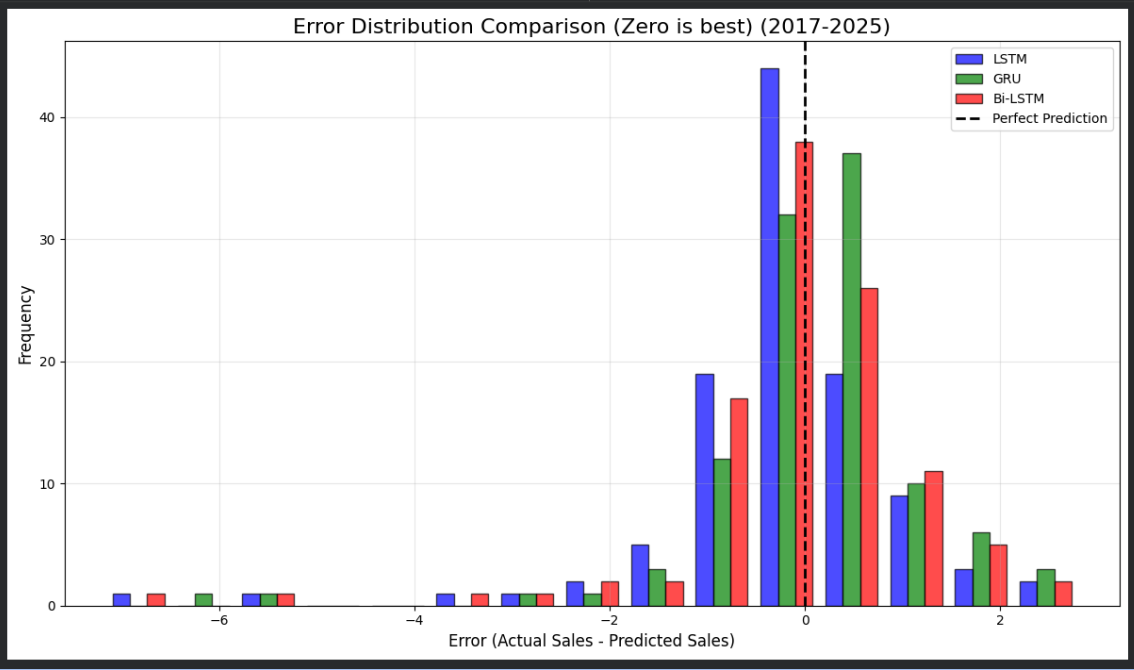


```
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
Generating future predictions for all models...
```




```
plt.tight_layout()
plt.show()
```



Evaluation and Comparison

Actual vs Predicted Sales (Last 5 Years):

The comparison plot shows actual sales values represented as bar charts, while predictions from LSTM, GRU, and Bi-LSTM are overlaid as line plots.

Observations:

- All three models successfully track the overall trend and seasonal behavior of the sales data.
- The Bi-LSTM model consistently follows the actual sales curve more closely during turning points and gradual trend changes.
- GRU demonstrates competitive performance with slightly smoother predictions.
- LSTM captures general trends well but shows marginal lag during abrupt changes

This indicates that bidirectional context improves temporal learning for economic time-series data.

Future Sales Forecast (Next 6 Months):

The future forecasting plot compares predictions for the next six months generated iteratively using the trained models.

Observations:

- All three models predict a **stable to slightly declining trend** in sales over the forecast horizon.
- LSTM forecasts slightly higher values compared to GRU and Bi-LSTM.
- Bi-LSTM produces more conservative and smoother forecasts, indicating better generalization.
- The consistency among the models increases confidence in the forecast reliability

This experiment demonstrates the ability of deep RNN models to generate **real future predictions** without access to future ground truth.

Residual Analysis (Error Distribution):

Residuals were calculated as:

Residual=Actual Sales−Predicted Sales

The error distribution histogram provides insight into model accuracy and bias.

Observations:

- Residuals for all models are centered around zero, indicating minimal systematic bias.
- **Bi-LSTM exhibits the narrowest error distribution**, suggesting lower variance and higher stability.
- GRU errors are slightly more spread but remain well-balanced.
- LSTM shows a wider distribution with occasional larger deviation

A tighter residual distribution implies better predictive reliability, especially for economic decision-making.

Overall Model Comparison:

Model	Trend Tracking Capability	Future Forecast Stability	Error Distribution Characteristics
LSTM	Good	Moderate	Wider error spread
GRU	Very Good	Stable	Balanced error distribution
Bi-LSTM	Excellent	Most stable	Narrowest error distribution

Conclusion

This study demonstrates that deep recurrent neural networks are highly effective for sales forecasting tasks. Among the evaluated models, Bi-LSTM outperforms LSTM and GRU in terms of trend alignment, future prediction stability, and residual error concentration. The bidirectional architecture enables the model to capture richer temporal patterns, making it particularly suitable for long-term economic forecasting.

Future work may include:

- **Incorporating external economic indicators**
- **Extending the forecast horizon**
- **Applying ensemble techniques for further accuracy improvements**

References

- OpenCV Python Documentation: https://docs.opencv.org/
- Use of AI Tools ChatGPT, Gemini for Some Reference
- Lecture Notes on Machine Learning by Professor Gnjatovic
- The dataset used in this experiment — **"TOTALSA: Total Vehicle Sales"** — was downloaded from the Federal Reserve Economic Data (FRED) portal: ➞ Source URL: https://fred.stlouisfed.org/series/TOTALSA This dataset represents the **monthly Total Vehicle Sales (Seasonally Adjusted Annual Rate)** for the United States and is commonly used for time-series forecasting benchmarks.